Jaculan, Warly France C.

4CS - B

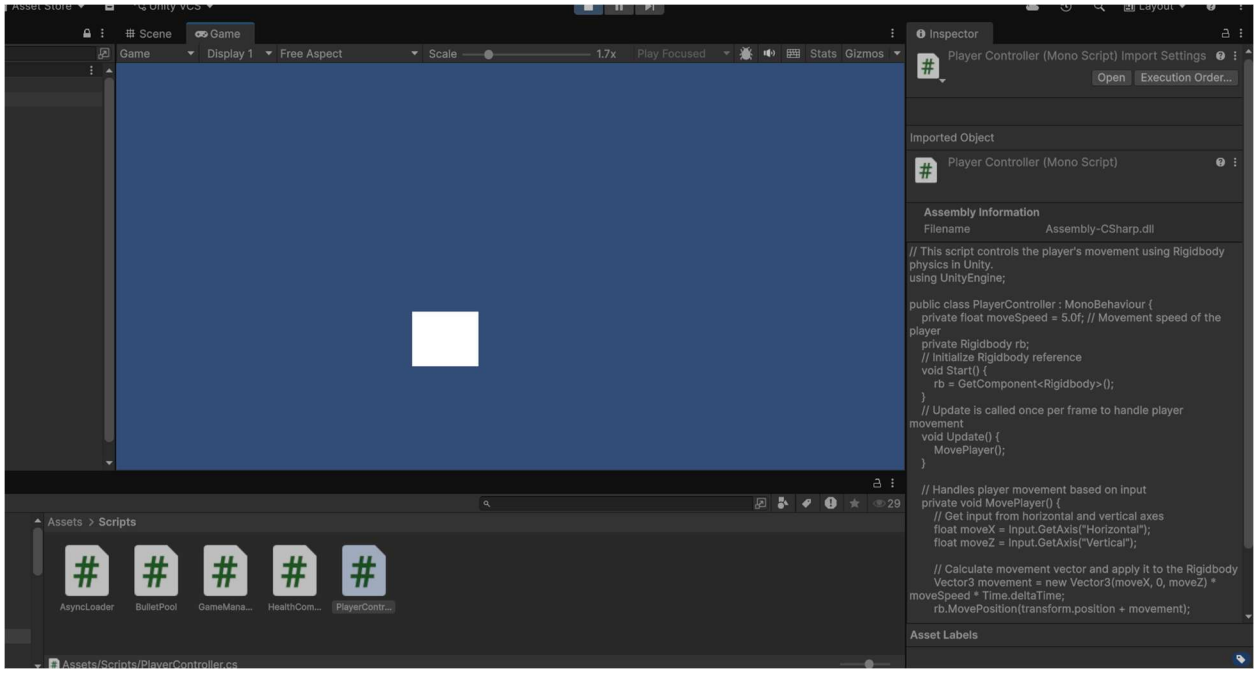
TASK 1

```
using UnityEngine;
public class PlayerController : MonoBehaviour {
private float moveSpeed = 5.0f; // Movement speed of the player
private Rigidbody rb;

void Start() {
rb = GetComponent<Rigidbody>();
}
void Update() {
MovePlayer();
}
// Handles player movement based on input
private void MovePlayer() {
float moveX = Input.GetAxis("Horizontal");
float moveZ = Input.GetAxis("Vertical");

Vector3 movement = new Vector3(moveX, 0, moveZ) * moveSpeed * Time.deltaTime;
rb.MovePosition(transform.position + movement);
}
}
```

With meaningful comments:

```
// This script controls the player's movement using Rigidbody physics in Unity.
using UnityEngine;
public class PlayerController : MonoBehaviour {
    private float moveSpeed = 5.0f; // Movement speed of the player
    private Rigidbody rb;
    // Initialize Rigidbody reference
    void Start() {
        rb = GetComponent<Rigidbody>();
    }
    // Update is called once per frame to handle player movement
    void Update() {
        MovePlayer();
    }

    // Handles player movement based on input
    private void MovePlayer() {
        // Get input from horizontal and vertical axes
        float moveX = Input.GetAxis("Horizontal");
```

```
        float moveZ = Input.GetAxis("Vertical");

        // Calculate movement vector and apply it to the Rigidbody
        Vector3 movement = new Vector3(moveX, 0, moveZ) * moveSpeed *
Time.deltaTime;
        rb.MovePosition(transform.position + movement);

    }
}
```

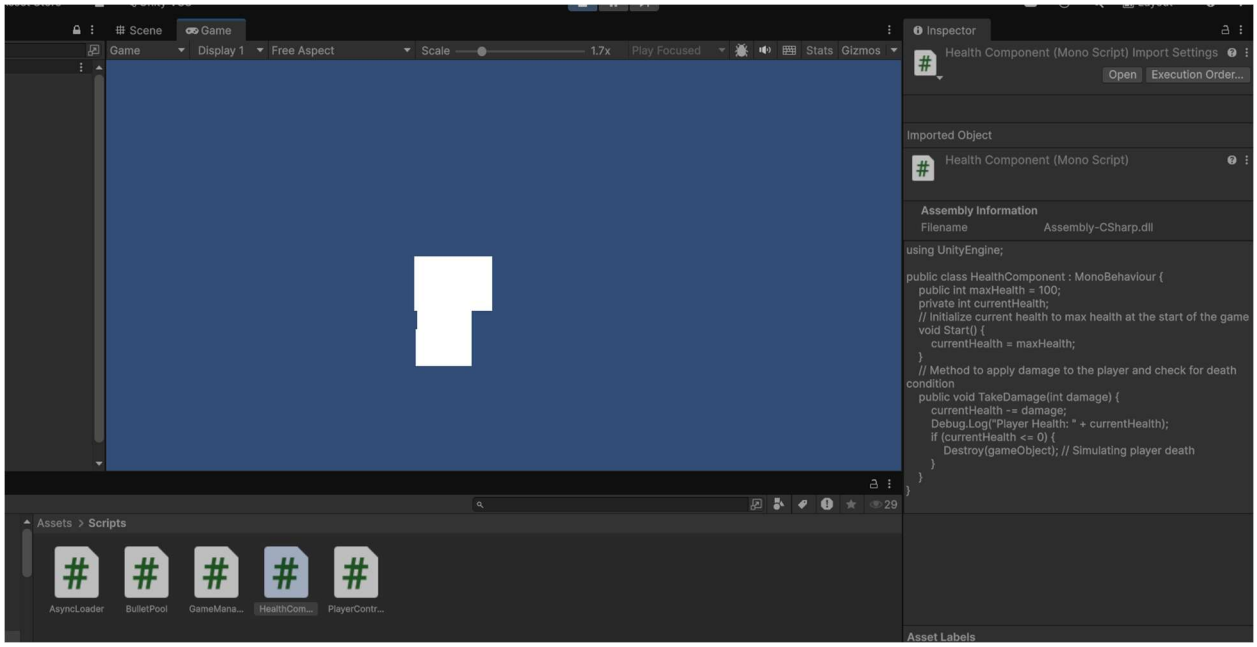


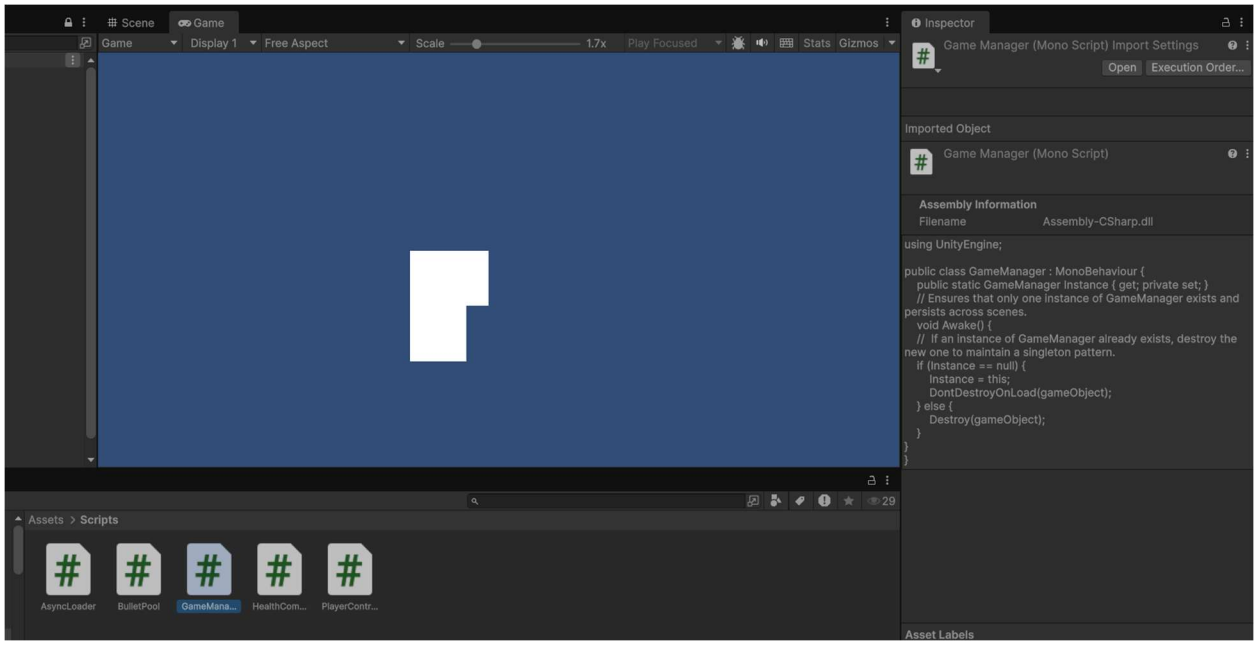
TASK 2

```
public class GameManager : MonoBehaviour {
public static GameManager Instance { get; private set; }

void Awake() {

if (Instance == null) {
Instance = this;
DontDestroyOnLoad(gameObject);
} else {
Destroy(gameObject);
}
}
}
```

Example: HealthComponent.cs (ECS Pattern)

```
using UnityEngine;

public class HealthComponent : MonoBehaviour {
public int maxHealth = 100;
private int currentHealth;

void Start() {
currentHealth = maxHealth;
}

public void TakeDamage(int damage) {
currentHealth -= damage;
Debug.Log("Player Health: " + currentHealth);
if (currentHealth <= 0) {
Destroy(gameObject); // Simulating player death
}
}
}
```


Scene
Game
Game
Display 1
Free Aspect
Scale
1.7x
Play Focused
Stats
Gizmos
Inspector
Health Component (Mono Script) Import Settings
Open
Execution Order...
Imported Object
Health Component (Mono Script)
Assembly Information
Filename
Assembly-CSharp.dll
using UnityEngine;
public class HealthComponent : MonoBehaviour {
public int maxHealth = 100;
private int currentHealth;
// Initialize current health to max health at the start of the game
void Start() {
currentHealth = maxHealth;
}
// Method to apply damage to the player and check for death condition
public void TakeDamage(int damage) {
currentHealth -= damage;
Debug.Log("Player Health: " + currentHealth);
if (currentHealth <= 0) {
Destroy(gameObject); // Simulating player death
}
}
}
Assets > Scripts
AsyncLoader
BulletPool
GameMana...
HealthCom...
PlayerContr...
Asset Labels


Scene
Game
Game
Display 1
Free Aspect
Scale
1.7x
Play Focused
Stats
Gizmos
Inspector
Game Manager (Mono Script) Import Settings
Open
Execution Order...
Imported Object
Game Manager (Mono Script)
Assembly Information
Filename
Assembly-CSharp.dll
using UnityEngine;
public class GameManager : MonoBehaviour {
public static GameManager Instance { get; private set; }
// Ensures that only one instance of GameManager exists and persists across scenes.
void Awake() {
// If an instance of GameManager already exists, destroy the new one to maintain a singleton pattern.
if (Instance == null) {
Instance = this;
DontDestroyOnLoad(gameObject);
} else {
Destroy(gameObject);
}
}
}
Assets > Scripts
AsyncLoader
BulletPool
GameMana...
HealthCom...
PlayerContr...
Asset Labels

TASK 3
BulletPool.cs

```
u sing System.Collections.Generic;
using UnityEngine;

public class BulletPool : MonoBehaviour
{
    [Header("Pool Settings")]
    [SerializeField] private GameObject bulletPrefab;
    [SerializeField] private int initialPoolSize = 20;
    [SerializeField] private bool canExpand = true;

    private Queue<GameObject> bulletPool;
    private Transform poolParent;

    void Awake()
    {
        // Pre-allocate queue capacity to avoid resizing (GC optimization)
        bulletPool = new Queue<GameObject>(initialPoolSize);

        // Keep hierarchy clean and reduce scene clutter
        poolParent = new GameObject("BulletPool").transform;
        poolParent.SetParent(transform);

        PrewarmPool();
    }

    private void PrewarmPool()
    {
        for (int i = 0; i < initialPoolSize; i++)
        {
            CreateBullet();
        }
    }

    private GameObject CreateBullet()
    {
        GameObject bullet = Instantiate(bulletPrefab, poolParent);
        bullet.SetActive(false);
        bulletPool.Enqueue(bullet);
        return bullet;
    }

    public GameObject GetBullet()
    {
        if (bulletPool.Count == 0)
        {
            if (canExpand)
                return CreateBullet();

            return null;
        }

        GameObject bullet = bulletPool.Dequeue();
```

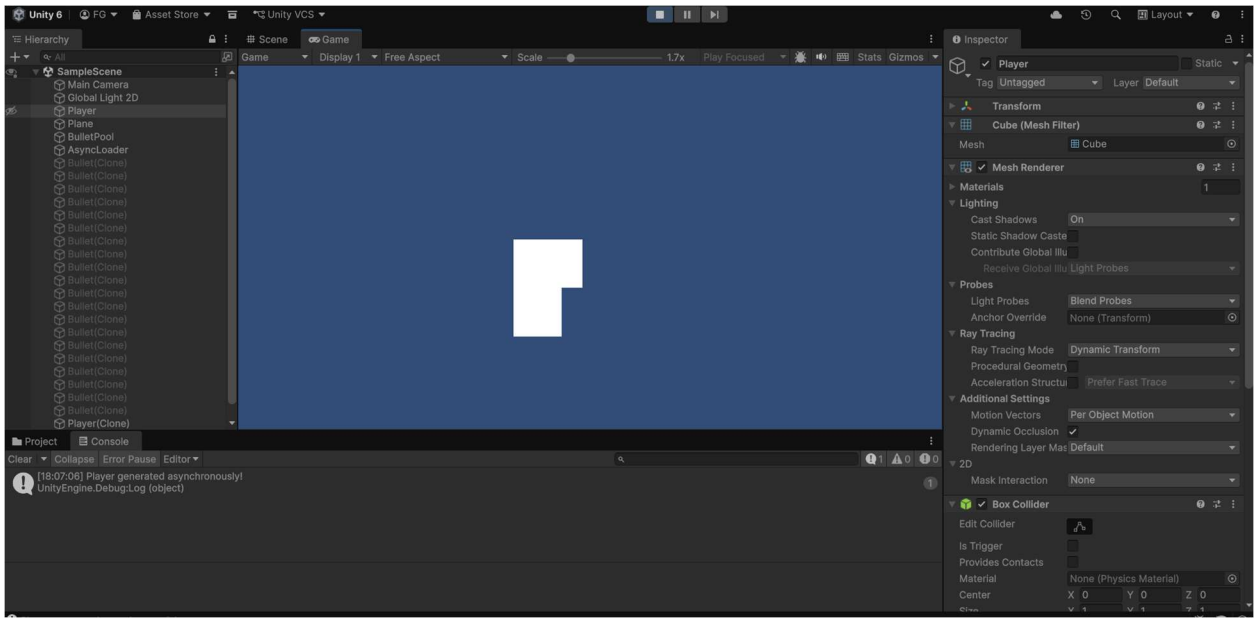
```
        bullet.SetActive(true);
        return bullet;
    }

    public void ReturnBullet(GameObject bullet)
    {
        bullet.SetActive(false);
        bulletPool.Enqueue(bullet);
    }
}
```

Script Explanation:

The BulletPool script improves performance by pre-instantiating a fixed number of bullet GameObjects at startup and reusing them instead of repeatedly calling Instantiate() and Destroy(), which are expensive operations that generate garbage and trigger CPU-intensive Garbage Collection (GC) spikes. By storing inactive bullets in a pre-sized Queue<GameObject>, the system avoids runtime memory reallocations and minimizes heap allocations, resulting in smoother frame rates—especially during rapid fire. Instead of destroying bullets, it simply disables them with SetActive(false) and returns them to the pool, eliminating memory fragmentation and reducing GC pressure. Efficient looping with a for loop during prewarming further ensures predictable performance, making this approach ideal for high-frequency object spawning scenarios like shooting mechanics in games.

TASK 4

AsynchLoader



AsynchLoader.cs:

```
using System.Collections;
using UnityEngine;

/// Demonstrates asynchronous loading using Resources.LoadAsync.
public class AsyncLoader : MonoBehaviour
{
    private IEnumerator Start()
    {
        ResourceRequest request = Resources.LoadAsync<GameObject>("Player");

        yield return request;

        // Check if the asset was loaded successfully before instantiating
        if (request.asset != null)
        {
            Instantiate(request.asset);
            Debug.Log("Player generated asynchronously!");
        }
        // Log an error if the asset failed to load
        else
        {
            Debug.LogError("Failed to load Player.");
        }
    }
}
```